

知光项目核心改造详细讲解（轻量版）

轻量改造方案 · 源码对照版 · 面试落地路线

生成日期	2026-09-09
默认方向	Java 后端岗位，兼顾少量 AI 应用后端能力
建议投入	7~8 个有效开发日
可信度边界	本文全部属于“建议改造”；只有真正编码、测试并保留结果后，才能写成已完成经历。

1. 先说结论：只做两项主改造和一项可选增强

当前项目已经有足够多的组件，不需要继续横向增加技术栈。最合理的做法是把已有链路补成闭环：

优先级	改造	预计投入	面试价值	是否必须
P0	发布状态约束 + 可靠 Outbox	2~3 天	事务、幂等、消息可靠性	必做
P0	Bitmap 计数的轻量对账与无 KEYS 重建	2~3 天	Redis、最终一致性、失败恢复	必做
P1	RAG 服务端过滤 + 简单引用 + 小评测集	1~2 天	AI 工程质量、可验证性	可选

三项改造共同遵循一个原则：不重写架构，只修复当前链路中最关键的正确性缺口，并留下可验证证据。

这次明确不做

- 不把单体拆成微服务；
- 不引入分库分表、复杂分布式事务框架或工作流引擎；
- 不把计数链路改造成 Redis Stream + Kafka 双消息系统；
- 不做完整推荐算法、关注流推拉结合或内容审核平台；
- 不搭建大而全的可观测平台，只加少量关键指标和结构化日志；
- 不追求“严格 Exactly Once”，目标是业务可幂等、失败可重试、结果可对账。

如果时间只有一周，完成前两项即可。RAG 增强适合面向 AI 应用后端岗位时再补。

2. 改造一：发布状态约束 + 可靠 Outbox

2.1 当前链路是什么

当前发布入口为：

```
POST /api/v1/knowposts/{id}/publish
-> KnowPostServiceImpl.publish
-> UPDATE know_posts SET status='published'
-> Redis [REDACTED] +1
-> INSERT outbox
-> Canal [REDACTED] binlog
-> Kafka canal-outbox
-> [REDACTED] Elasticsearch
```

相关源码：

- src/main/java/com/tongji/knownpost/service/impl/KnowPostServiceImpl.java
- src/main/resources/mapper/KnowPostMapper.xml
- src/main/java/com/tongji/relation/outbox/CanalKafkaBridge.java
- src/main/java/com/tongji/search/outbox/CanalOutboxConsumerSearch.java
- db/schema.sql

2.2 真正需要解决的三个问题

问题一：重复发布会重复增加发文数

当前发布 SQL 只限制 id + creator_id，没有限制原状态。第一次发布后再次调用，SQL 仍可能更新成功，随后作者发文数再次加一。

这不是一定要引入复杂状态机的问题。当前业务只有草稿、发布和删除三个主要状态，用 SQL 条件约束就足够。

问题二：Outbox 失败仍可能提交业务事务

KnowPostServiceImpl 和 RelationServiceImpl 都会捕获并吞掉 Outbox 插入异常。这样可能出现：业务表已经变化，但没有对应事件，搜索或关系投影永远收不到更新。

Outbox 的价值正是“业务记录和事件在同一个本地事务提交”。因此这里不能吞异常；Outbox 写失败时，应让整个 MySQL 事务回滚。

问题三：Canal 在 Kafka 真正发送成功前就确认批次

KafkaTemplate.send() 是异步调用。当前桥接器发起发送后直接 connector.ack(batchId)，如果发送 Future 随后失败，Canal 位点已经推进，消息可能丢失。

2.3 最小改造目标

只实现以下四条规则：

- 只有 draft 可以进入 published；
- 重复发布同一篇内容时，不重复计数、不重复制造业务副作用；
- 内容更新与 Outbox 必须在同一个 MySQL 事务成功或失败；
- Canal 只有在整批 Kafka 发送成功后才确认位点。

不增加审批流、审核中、驳回、定时发布等状态。这些功能不是当前求职展示的必要条件。

2.4 第一步：用条件更新实现轻量状态机

将 Mapper 中的发布 SQL 收紧为：

```
UPDATE know_posts
SET status = 'published', publish_time = NOW(), update_time = NOW()
WHERE id = #{id}
  AND creator_id = #{creatorId}
  AND status = 'draft'
  AND content_object_key IS NOT NULL;
```

这里的关键不是 SQL 写法，而是业务含义：

- status = 'draft' 防止重复发布和已删除内容复活；
- content_object_key IS NOT NULL 防止没有正文就发布；
- 返回影响行数为 1，说明本次请求真正完成了状态转换；
- 返回 0 时再查询当前记录，区分“已经发布”“无权限”“内容不存在”和“正文未上传”。

服务层可以采用下面的处理逻辑：

```
updated == 1
-> Outbox

updated == 0
->
-> published
->
-> draft
-> deleted
```

这样已经能覆盖绝大多数网络重试和用户重复点击，不需要马上建设通用幂等表。

2.5 第二步：让 Outbox 真正参与事务

改造前的危险写法：

```
try {
    outboxMapper.insert(...);
} catch (Exception e) {
    log.warn(...);
}
```

建议改为直接插入，让异常继续抛出：

```
long eventId = idGen.nextId();
String payload = objectMapper.writeValueAsString(...);
outboxMapper.insert(eventId, "knowpost", id, "KnowPostPublished", payload);
```

因为 publish() 已有 @Transactional，此时会得到清晰的事务结果：

```
know_posts ■■■■ + outbox ■■■■ -> ■■■■  
know_posts ■■■■ + outbox ■■■■ -> ■■■■
```

现有 outbox.id 就可以作为稳定的 eventId，无需新建消息表。建议让 Canal 桥接消息同时携带：

- id：事件 ID；
- aggregate_type：聚合类型；
- aggregate_id：业务对象 ID；
- type：事件类型；
- payload：业务数据。

当前桥接器只提取了 payload，应把以上字段一起提取出来，方便日志追踪和消费去重。

2.6 第三步：Kafka 成功后再确认 Canal 位点

最小实现不必上复杂事务消息。桥接器按批次收集发送结果，设置一个短超时即可：

```
■■ Canal batch  
-> ■■■■ Outbox ■  
-> kafka.send(eventId, message)  
-> ■■■■■■ Future ■■■■■■ 3 ■  
-> ■■■■■■ack(batchId)  
-> ■■■■■■rollback(batchId)■■■■■■■■■■
```

Kafka 消息 Key 使用 aggregate_type + ':' + aggregate_id，可使同一业务对象的事件进入同一分区，便于保持顺序。

这里要接受“至少一次”语义：如果 Kafka 已成功但进程在 Canal ack 前崩溃，重启后消息会重复投递。因此消费者必须幂等。

2.7 消费幂等做到什么程度就够了

这次不建设通用 Inbox 平台，只按消费者类型处理：

- 搜索消费者：ES 文档 ID 固定为知文 ID，upsert 和 delete 本身可以重复执行；
- 关系消费者：以 outbox.id 作为去重 ID，Redis SETNX dedup:outbox:{eventId}，TTL 设置 1 ~ 7 天；
- 消费异常不要吞掉，也不要 ack，让 Kafka 重试；
- 日志至少打印 eventId、aggregateId、eventType 和异常原因。

Redis 去重不是永久的严格一次语义，但作为个人项目已经能清楚展示“重复消息是正常现象，消费端按业务幂等处理”。若未来真有生产要求，再升级为 MySQL Inbox 唯一索引，不需要现在就做。

2.8 缓存与 RAG 何时触发

Redis、Caffeine 和 RAG 不属于 MySQL 事务，建议在事务提交后执行：

```
MySQL ██████
-> AFTER_COMMIT ███
-> ██████ Feed ██████
-> ██████████
-> ██████ RAG ███
```

可用 @TransactionalEventListener(phase = AFTER_COMMIT) 实现。失败时记录日志并依靠 TTL、计数重建或首次问答时的 ensureIndexed 兜底。

这里不要宣称缓存和 RAG 与 MySQL 强一致。它们是允许延迟、可以修复的派生数据。

2.9 建议修改的文件

文件	最小修改
KnowPostMapper.xml	发布 SQL 增加 status='draft' 与正文条件
KnowPostServiceImpl.java	区分首次发布/重复发布；Outbox 异常不再吞掉
CanalKafkaBridge.java	提取 Outbox 完整字段；等待 Kafka Future；失败 rollback
CanalOutboxConsumerSearch.java	异常不 ack，补充 eventId 日志
RelationEvent.java / 关系消费者	增加 eventId，并按 eventId 去重
新增提交后监听器	事务提交后处理缓存、计数和 RAG 预索引

2.10 最少要写的测试

- 同一内容连续发布两次，状态只转换一次，作者发文数只增加一次；
- 草稿没有正文时不能发布；
- 模拟 Outbox 插入失败，know_posts 状态仍为 draft；
- 模拟 Kafka Future 失败，Canal 批次执行 rollback 而不是 ack；
- 同一 Outbox 消息重复消费两次，ES 结果不变，关系计数不重复增加。

不必一次搭建全套中间件。前 3 个可用 MySQL/Testcontainers，后 2 个先用 Mock 验证确认语义；有余力再补 Kafka 集成测试。

2.11 面试时怎么讲

原发布接口只按内容 ID 更新，重复请求可能重复增加发文数；同时 Outbox 插入和 Kafka 发送异常被吞后，业务事实与搜索投影可能永久不一致。我将发布入口收敛为 draft -> published 条件更新，把 Outbox 失败纳入本地事务回滚，并让 Canal 等待 Kafka 发送成功后再确认位点。链路采用至少一次

投递，搜索通过固定文档 ID 幂等覆盖，关系事件按 eventId 去重。

2.12 完成标准

- 能画出发布事务和异步投影数据流；
- 能演示重复发布不会重复计数；
- 能通过测试证明 Outbox 失败时业务事务回滚；
- 能解释为什么 Canal/Kafka 是至少一次，而不是严格一次；
- 有一条失败日志能通过 eventId 追到具体内容。

3. 改造二：Bitmap 计数的轻量对账与无 KEYS 重建

3.1 当前链路是什么

```
■■/■■■■■
-> Lua: GETBIT + SETBIT
-> Kafka counter-events
-> Redis Hash ■■ delta
-> ■■■■■■■■ SDS
-> Feed/■■■■■ SDS

SDS ■■■■■■
-> Redisson ■■■■■■
-> KEYS ■■ Bitmap ■■
-> BITCOUNT ■■ SDS
```

当前设计最大的优点是：Bitmap 保存用户行为事实，SDS 只是可重建快照。改造时应保留这一点。

3.2 不采用 Redis Stream 的原因

把 SETBIT + XADD 放进同一个 Lua 确实能解决 Bitmap 与 Kafka 的双写窗口，但会新增 Stream 消费组、待确认列表、转发去重和运维指标。对于个人项目，这条链路容易越改越重。

更轻量的方案是：Kafka 保留为实时快路径，同时在 Bitmap 变化时原子标记“该实体需要对账”。即使 Kafka 增量偶尔丢失，后台任务仍能从 Bitmap 事实修正 SDS。

3.3 最小目标

- 不再用 KEYS bm:* 和 KEYS agg:* 扫描 Redis；
- 每次真实状态变化都留下可恢复标记；
- Kafka 仍负责秒级更新，对账任务只负责最终纠偏；
- 对账时不因并发新写入误删最新脏标记。

3.4 扩展现有 Bitmap Lua

当前 Lua 只操作一个 Bitmap Key。建议增加两个索引 Key：

```
KEYS[1] = bm:like:knowpost:1001:3
KEYS[2] = bm:shards:like:knowpost:1001
KEYS[3] = counter:dirty

ARGV[1] = bitOffset
ARGV[2] = add/remove
ARGV[3] = chunkId
ARGV[4] = dirtyMember like|knowpost|1001
ARGV[5] = 
```

只有状态真正变化时，Lua 额外执行：

```
redis.call('SADD', KEYS[2], ARGV[3])
redis.call('ZADD', KEYS[3], ARGV[5], ARGV[4])
```

这样一次原子操作同时完成：

- 修改用户行为事实；
- 记录该实体有哪些 Bitmap 分片；
- 标记该指标需要对账。

Kafka 发送仍在 Lua 之后执行，它是低延迟更新 SDS 的快路径；即使发送失败，counter:dirty 也已经存在。

3.5 用分片索引替代 KEYS

当前重建使用：

```
KEYS bm:{metric}:{entityType}:{entityId}:*
```

改造后直接读取：

```
SMEMBERS bm:shards:{metric}:{entityType}:{entityId}
```

随后根据 chunk ID 拼出精确 Bitmap Key，再用 Pipeline 执行 BITCOUNT。这样不会扫描 Redis 全局键空间。

分片集合只保存数字 ID，数据量通常远小于 Bitmap 本身。即使集合偶尔多一个已删除分片，BITCOUNT 空 Key 返回 0，也不会影响正确性。

3.6 增加一个小型对账任务

任务每 5 秒执行一次，每批最多处理 100 个实体指标：

```

■■ counter:dirty ■ score <= now - 5s ■■ 100 ■
-> ■■■■■■■■
-> Pipeline BITCOUNT
-> ■ Redisson ■■■■■■■■ SDS ■■
-> ■■■■■■■■■■■■■■■■■■■■

```

为什么等待 5 秒：让正常 Kafka 聚合先完成，避免每次点赞都立即全量重算。持续热门的内容会不断刷新 score，对账只在短暂安静后执行。

3.7 删除脏标记时要避免竞态

假设对账任务读取到旧 score 后开始 BITCOUNT，此时又来了一个点赞，Lua 会更新 counter:dirty 的 score。若任务结束后直接 ZREM，就会把新的脏标记误删。

解决方法是用一个很小的 Lua 做 compare-and-remove：

```

local current = redis.call('ZSCORE', KEYS[1], ARGV[1])
if current and current == ARGV[2] then
    return redis.call('ZREM', KEYS[1], ARGV[1])
end
return 0

```

任务只有在 score 仍等于自己读取到的值时才删除。若期间发生新写入，score 已变化，脏标记保留到下一轮。

这比引入分布式任务平台简单很多，但足以体现并发竞态处理意识。

3.8 聚合桶也不要使用 KEYS

CounterAggregationConsumer.flush() 当前扫描 KEYS agg:*。可以增加一个固定集合 counter:agg:dirty：

```

■■ Kafka ■■
-> Lua: HINCRBY aggKey field delta
-> SADD counter:agg:dirty aggKey
-> ■■■ ack Kafka

■■ flush
-> ■ counter:agg:dirty ■■■■■■
-> ■■ Lua ■■■■■SDS ■ delta■Hash ■ delta
-> Hash ■■■■■■ Lua ■■■■■■ aggKey

```

当前代码先执行 SDS 加法 Lua，再执行 Hash 扣减 Lua；若进程恰好在两次调用之间退出，下一轮会把同一 delta 再加一次。建议把两步合并为一个脚本，并传入三个 Key：

```
KEYS[1] = cntKey
KEYS[2] = aggKey
KEYS[3] = counter:agg:dirty
```

脚本在 Redis 内原子完成：读取聚合字段、更新 SDS、从 Hash 扣除本次 delta；字段归零时删除字段，整个 Hash 为空时再 SREM dirty 集合。Redis 串行执行 Lua，因此并发消费者若在脚本之后写入，会重新 SADD；若在脚本之前写入，Hash 不为空，dirty 成员不会被误删。

不要先 SPOP 再处理，否则进程在刷写前崩溃会丢失待处理索引。先读取成员，只有原子折叠完成且 Hash 为空后才删除成员。

3.9 对账任务与 Kafka 会不会重复计算

不会使用同一种更新方式：

- Kafka 聚合是对 SDS 做 +delta，追求低延迟；
- 对账任务从 Bitmap 重新计算总数，然后覆盖 SDS 对应字段，追求最终正确。

对账写入需要持有现有的 lock:sds-rebuild:{etype}:{eid} 锁，避免两个重建任务同时覆盖。对账结束后，如果聚合 Hash 中仍有未刷写增量，不应盲目删除；可以让下一轮 Kafka flush 或下一轮对账继续收敛。

3.10 建议修改的文件

文件	最小修改
CounterKeys.java	增加 shard set、dirty ZSet、aggregation dirty set Key
CounterServiceImpl.java	扩展 toggle Lua；重建从分片集合读取
CounterAggregationConsumer.java	聚合时登记 dirty aggKey；合并 SDS 加法与 Hash 扣减 Lua；flush 不再使用 KEYS
新增 CounterReconcileJob.java	按批次从 Bitmap 覆盖校正 SDS
CounterEventProducer.java	发送失败记录实体、指标和异常；不再静默吞掉

3.11 最少要写的测试

- 重复点赞不会产生新的 dirty score，也不会重复计数；
- 真实状态变化会同时写 Bitmap、分片集合和 dirty ZSet；
- 模拟 Kafka 事件完全丢失，对账后 SDS 与 Bitmap 一致；
- 对账期间发生新点赞，compare-and-remove 不会删除新脏标记；
- Redis 中存在大量无关 Key 时，重建流程不调用 KEYS；
- 聚合桶刷写失败后，counter:agg:dirty 中的成员仍然保留；
- 原子折叠脚本执行后，SDS 增量与 Hash 剩余 delta 始终对应，不存在只完成一半的状态。

优先用 Testcontainers Redis 完成这些测试。它比单纯 Mock 更能证明 Lua、Bitmap、ZSet 和 Pipeline 的组合行为。

3.12 只加四个指标

不搭建复杂大盘，只记录：

- counter_event_send_fail_total：Kafka 发送失败次数；
- counter_dirty_size：等待对账的实体指标数量；
- counter_reconcile_total{result}：对账成功/失败次数；
- counter_reconcile_diff：对账前后差值。

面试时真正有价值的是展示一次故障实验：人为让 Kafka 不可用，继续点赞，然后恢复并观察对账将 SDS 修正。

3.13 面试时怎么讲

Bitmap 是用户行为事实，Kafka 和 SDS 是实时聚合链路。原实现存在 Bitmap 已更新但 Kafka 增量失败的窗口，而且重建依赖 Redis KEYS。我保留 Kafka 作为秒级快路径，在同一个 Bitmap Lua 中登记分片集合与 dirty ZSet；后台按安静窗口从 Bitmap 覆盖校正 SDS，并用 score compare-and-remove 避免并发写入时误删新脏标记。这样没有引入新的消息系统，也能实现可解释的最终一致性。

3.14 完成标准

- 代码中不再出现计数相关的 redis.keys(...);
- Kafka 停止时点赞状态仍能写入，恢复后计数最终正确；
- 能解释“快路径增量 + 慢路径覆盖对账”的职责差异；
- 有至少一个真实 Redis 集成测试覆盖并发脏标记；
- 有一份故障实验记录，包含操作步骤和修正前后数据。

4. 改造三：RAG 服务端过滤 + 简单引用 + 小评测集

这项是可选增强。若主投 Java 后端岗位，前两项完成后再做；不要为了 AI 亮点影响事务和计数改造的完成度。

4.1 当前问题

当前 RagQueryService 先从全局向量索引取 $\max(\text{topK} * 3, 20)$ 条，再由 Java 根据 postId 过滤。

当文章数量增大时，全局 Top 20 可能都来自其他文章。即使目标文章已有正确切片，过滤后仍可能得到空上下文。

4.2 第一步：在向量查询阶段过滤 postId

项目使用的 Spring AI 1.0.3 已支持 `SearchRequest.Builder.filterExpression(String)`。可直接改为：

```
SearchRequest request = SearchRequest.builder()
    .query(question)
    .topK(topK)
    .filterExpression("postId == '" + postId + "'")
    .build();

List<Document> docs = vectorStore.similaritySearch(request);
```

postId 来自 long，不接收任意用户字符串，因此这里没有表达式注入问题。改造后不再需要全局宽召回再过滤，候选集从一开始就限定在当前文章。

4.3 第二步：做最简单的引用，不改 SSE 协议

不要马上设计复杂的结构化事件协议。先把召回片段格式化为：

```
[███1 | chunkId=1001#3 | position=3]
██████.....

[███2 | chunkId=1001#5 | position=5]
██████.....
```

Prompt 增加一句:

[REDACTED] [REDACTED]

前端仍按现有 SSE 文本流展示，不需要同步修改事件协议。用户至少能够看到答案依据了哪个片段。

这不是严格的引用校验，因为模型可能标错片段；但作为第一版足够。后续真的需要交互式引用时，再升级为结构化 retrieval/token/citation/done 事件。

4.4 第三步：建立 20 条小评测集

不需要一开始做 100 条，也不必引入复杂评测平台。建立一个 JSON 文件即可：

[illegible]

```
}  
]
```

第一版只统计三个指标：

- Hit@3：前 3 个召回片段是否包含标准片段；
- 引用覆盖率：回答中的主要结论是否带 [片段N]；
- P95 延迟：从请求开始到回答结束的耗时。

不要急着加入 LLM-as-a-Judge。先用人工标注片段证明检索正确，再考虑 Faithfulness 等模型评估。

4.5 建议修改的文件

文件	最小修改
RagQueryService.java	使用 filterExpression；保留 Document 元数据；上下文加入片段编号
RagIndexService.java	确认 postId/chunkId/position/title 元数据稳定，不必重写切片算法
src/test/resources/rag-eval.json	保存约 20 条固定问题与标准片段
新增一个评测测试类	输出 Hit@3、引用覆盖率和延迟摘要

4.6 这次不要做什么

- 不做多路 Query Rewrite；
- 不接入独立 reranker；
- 不做多 Agent 自反思；
- 不做复杂知识图谱；
- 不立即重写切片算法；
- 不把 LLM 评分当成唯一正确答案。

4.7 最少要写的测试

- 两篇文章都包含相似内容时，只召回指定 postId 的切片；
- 目标文章没有相关内容时，返回“不确定”而不是使用其他文章；
- 回答 Prompt 中包含 chunkId/position 与片段编号；
- 评测脚本可以重复执行并输出相同格式的结果。

4.8 面试时怎么讲

原 RAG 在全局向量 Top N 后才按 postId 过滤，文章增多时可能被其他文章占满候选集。我把过滤下推到 Spring AI 的 VectorStore 查询阶段，并保留 chunkId、position 等元数据生成片段引用；同时建立 20 条固定问答集，用 Hit@3、引用覆盖率和 P95 延迟做回归。改造重点是证明检索边界和质量，而不是继续堆模型组件。

4.9 完成标准

- 服务端过滤测试通过，无法发生跨文章召回；
- 页面上的回答能看到 [片段N]；
- 20 条评测集可一键执行；
- 简历中的任何准确率或延迟数字都能从评测输出复现。

5. 推荐实施顺序

5.1 七到八天版本

天数	工作	当天必须留下的产物
第 1 天	发布 SQL 状态约束、重复发布语义	状态转换测试
第 2 天	Outbox 异常回滚、提交后事件	事务失败测试
第 3 天	Canal 等待 Kafka、失败 rollback、eventId 日志	桥接确认语义测试
第 4 天	Bitmap Lua 登记 shard/dirty	Redis Lua 集成测试
第 5 天	对账任务与 compare-and-remove	Kafka 丢失后的修复测试
第 6 天	聚合桶去除 KEYS、补四个指标	故障实验记录
第 7 天	RAG postId 过滤与片段编号	跨文章污染测试
第 8 天	20 条评测集、整理面试图与数据	评测报告、最终 README

若时间不足，第 7~8 天可以整体删除，不影响 Java 后端主线。

5.2 每项改造只保留一个核心故事

- 发布链路：重复请求和消息失败下，如何保证业务事实与事件不分离；
- 计数链路：实时增量可能丢失时，如何依靠事实源最终纠偏；
- RAG 链路：如何证明只检索指定文章，并用固定评测集防止回归。

面试时不要把所有组件都讲一遍。每个故事讲清“问题、方案、失败场景、测试证据”即可。

6. 建议简历功能点（完成对应改造后可写）

以下内容现在仍是建议写法，完成代码、测试和实验后再放进简历：

- 收敛内容发布状态转换，在 SQL 条件更新中限制 draft -> published 并处理重复请求；将业务记录与 Outbox 置于同一事务，Outbox 失败时整体回滚，避免重复发文计数和搜索事件丢失。
- 修正 Canal 到 Kafka 的确认语义，等待批次内消息发送成功后再推进 Canal 位点，失败时回滚重试；使用稳定 eventId 和幂等覆盖处理至少一次投递产生的重复消息。
- 为 Bitmap 计数增加轻量对账链路，在 Lua 原子切换状态时登记分片集合与 dirty ZSet，以 Kafka 聚合作为实时快路径、Bitmap 覆盖重建作为修复慢路径，消除全局 KEYS 扫描并处理并发脏标记竞态。
- 将文章级 RAG 的 postId 过滤下推到向量检索阶段，保留 chunkId 与位置元数据生成片段引用，并用固定问答集持续验证 Hit@3、引用覆盖率和 P95 延迟。

简历最终选择其中 2~3 条即可，不要四条全部堆上去。

7. 高频追问与简短回答

Q：为什么不直接上完整状态机框架？ A：当前只有草稿、发布、删除三个核心状态，用条件更新已经能解决重复发布和非法转换。框架会增加学习和维护成本，但没有带来等比例业务价值。

Q：为什么 Outbox 插入失败要回滚发布？ A：搜索索引和其他投影依赖该事件。如果业务事实提交但事件缺失，下游无法知道这次变化；让二者在同一 MySQL 事务成功或失败，恢复路径更清晰。

Q：等待 Kafka Future 会不会降低吞吐？ A：等待发生在独立 Canal 桥接线程，不阻塞用户发布请求；批次并行发送后统一等待，换取明确的位点确认语义。超时后整批重试，消费者负责幂等。

Q：计数为什么不用 Redis Stream 一次解决？ A：Stream 可以做，但会新增一条消息消费链路。当前 Bitmap 已经是事实源，只要原子登记 dirty 标记并周期对账，就能以更小改造获得最终一致性；Kafka 继续承担实时更新。

Q：覆盖重建会不会把刚到的增量覆盖掉？ A：重建在实体锁内按 Bitmap 当前事实计算总值；dirty 标记只有 score 未变化时才删除。若重建期间有新写入，标记会保留，下一轮继续校正。

Q：RAG 为什么只做 20 条评测？ A：第一阶段目的是建立可重复基线而不是追求统计完美。20 条人工标注足以暴露跨文章污染、召回失败和引用缺失；后续根据错误类型再扩充数据集。

8. 最终自检

- 是否真的删除了 Outbox 相关的吞异常代码？
- 重复发布是否有测试证明不会重复产生副作用？
- Canal 是否在 Kafka Future 成功后才 ack？

- 计数代码是否已不再使用 `redis.keys(...)`?
- Kafka 丢失增量后，SDS 是否能由 Bitmap 自动修正?
- RAG 查询是否在 VectorStore 阶段过滤 `postId`?
- 每项改造是否至少有一个失败场景测试?
- 简历上的数字是否能从测试或评测输出复现?

完成前两项，项目已经足以形成一条扎实的 Java 后端面试主线。第三项只用于增强 AI 应用后端区分度，不应反过来扩大项目范围。